

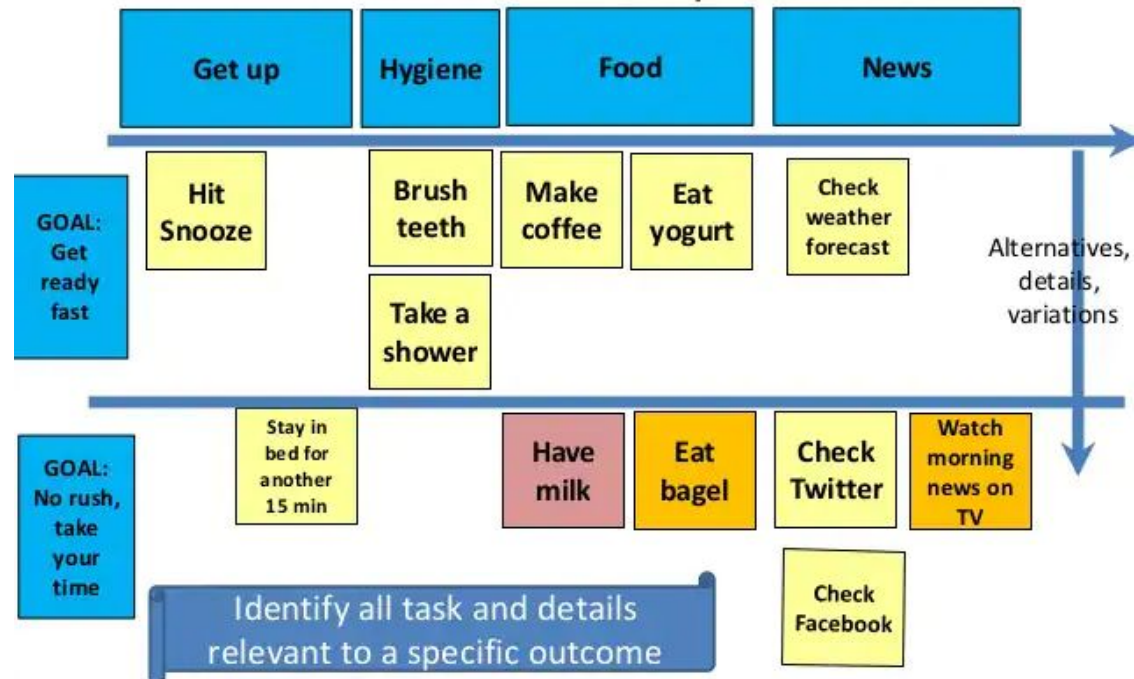
Conceptual Thinking

Story Mapping, Wireframes & Use case diagramma's,
Activity Diagram

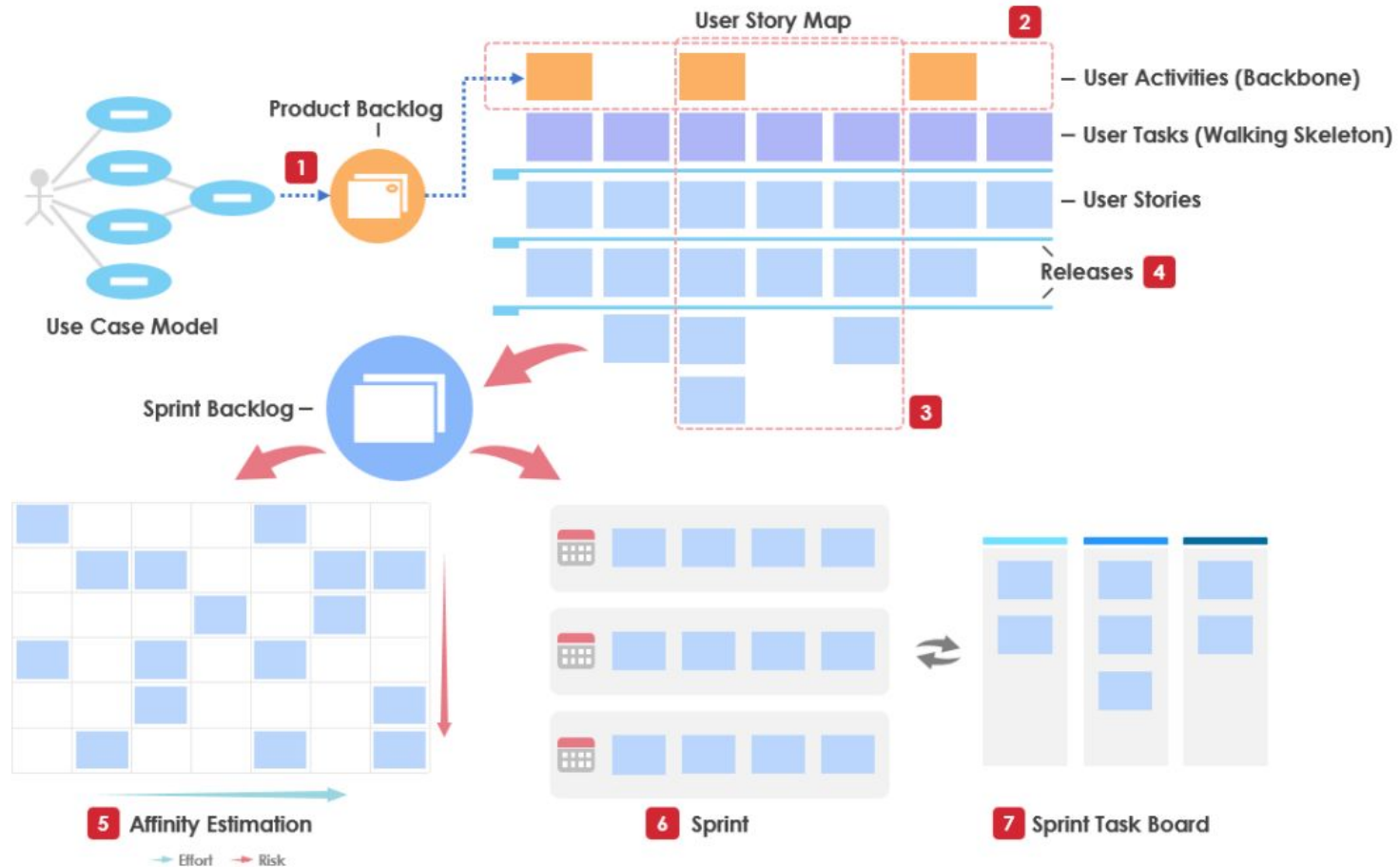
Story Map: General

Story Map

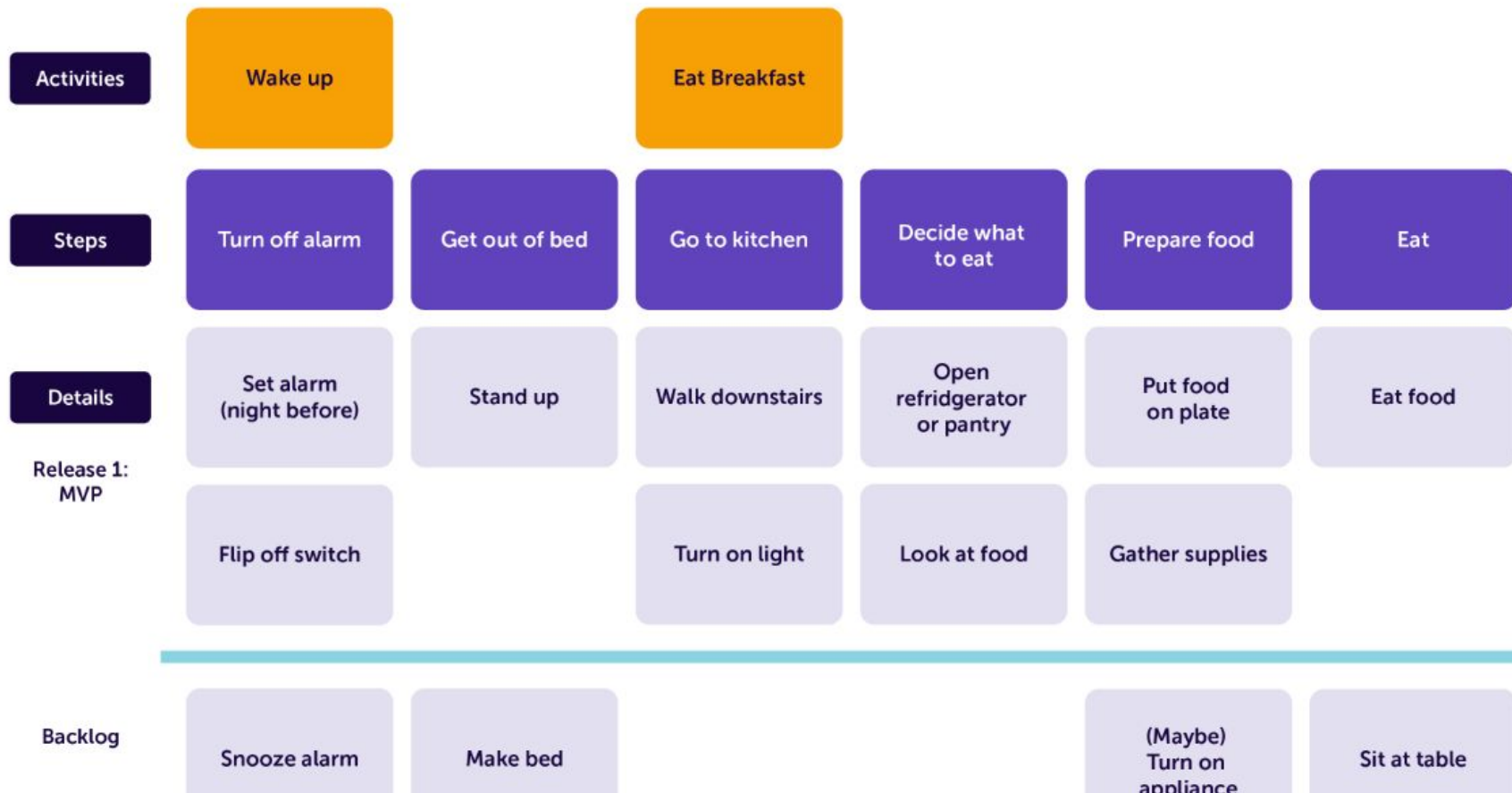
A story map organizes user stories according to a narrative flow that presents the big picture of the product. This reduces the risk that a large number of highly specific user stories distracts the project team from achieving the product's key objectives.



Story Map



Story Map



Tools

<https://miro.com/templates/user-story-map/>

<https://www.justinmind.com/blog/user-story-mapping/>

<https://www.mural.co/templates/user-story-map>

<https://www.avion.io/what-is-user-story-mapping/>

<https://www.visual-paradigm.com/tour/agile-development-tools/user-story-map.jsp>

How to do Story mapping?

1. Determine the journey

- a. Determine the TOP goals for your product/solution.
- b. Answering the questions:
 - **What?** What problem are you trying to solve? What product do you want to build or feature do you want to add?
 - **Who?** Is there a specific user or user subgroup you're building for? What benefits does each of them get from what you're building?
 - **Why?** What's the benefit to your company of creating this feature or product? How will it add value to your business' bottom line for users?

Snakes & Ladders



Goal :

Have a group of 2-4 people play a single game of S&L on a single device according to the S&L rules.

How to do Story mapping?

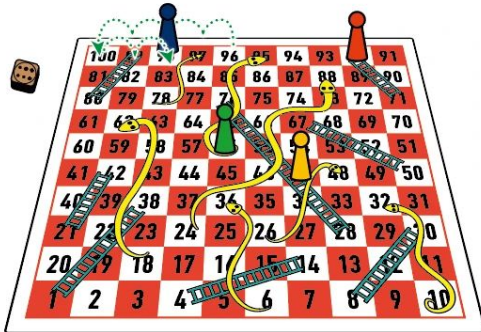
2. Build the “backbone” of your story

- a. Either start with the scenarios of your use cases, or write the user path described in tasks or steps from start to finish.
- b. Don't include too many details.

Backbone

1. Starting the game
2. For all players:
 - a. note the player's name
 - b. assign a random color to the player's pawn
3. Initialize the board (all pawns at the beginning)
4. For each player:
 - a. throw the dice
 - b. move the pawn according to the dice result and the rules
5. Repeat until one player reaches the finish position (100)
6. Display the final result on the screen
7. End of program

Snakes & Ladders

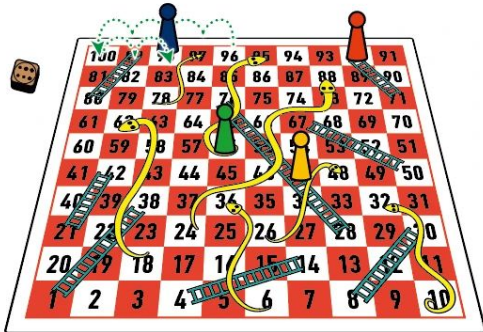


How to do Story mapping?

3. Identify and group activities

- a. Note some common themes. Many of these steps likely work toward a common goal. In user story mapping, we call these activities.

Snakes & Ladders



Backbone

1. Starting the game
2. For all players:
 - a. note the player's name
 - b. assign a random color to the player's pawn
3. Initialize the board (all pawns at the beginning)
4. For each player:
 - a. throw the dice
 - b. move the pawn according to the dice result and the rules
5. Repeat until one player reaches the finish position (100)
6. Display the final result on the screen
7. End of program

S
E
T
U
P

P
L
A
Y

E
N
D

How to do Story mapping?

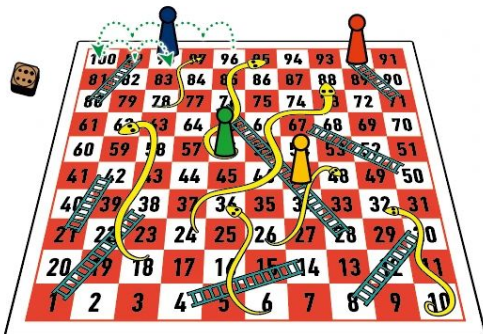
4. Split up

- Distribute the available must-have user stories across the backbone activities.
- Divide the stories into smaller tasks.
- Don't shoot down ideas yet.

Split up:

- draw the outline of the board
- draw the cells
 - show the cell number
 - display whether a ladder starts in cell ...
- ...
- move the pawn down the snake
- nice to have - play online/mobile/in Minecraft...
- ...
- nice to have - let the user change the color/shape of the pawn
 - provide a list of colors/shapes to choose from
 - record the choice
 - use that choice in the pawn display

Snakes & Ladders



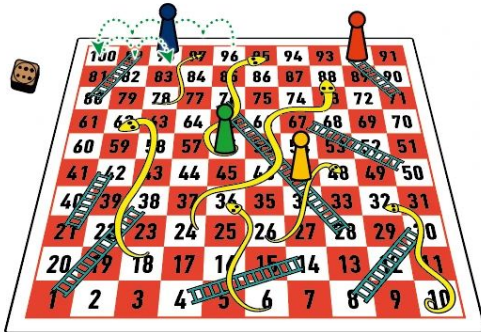
How to do Story mapping?

5. Prioritize

- Move user stories/tasks up and down to visually display the priority (top is high priority, down is low priority)

Adding more players is less of a priority than drawing the board

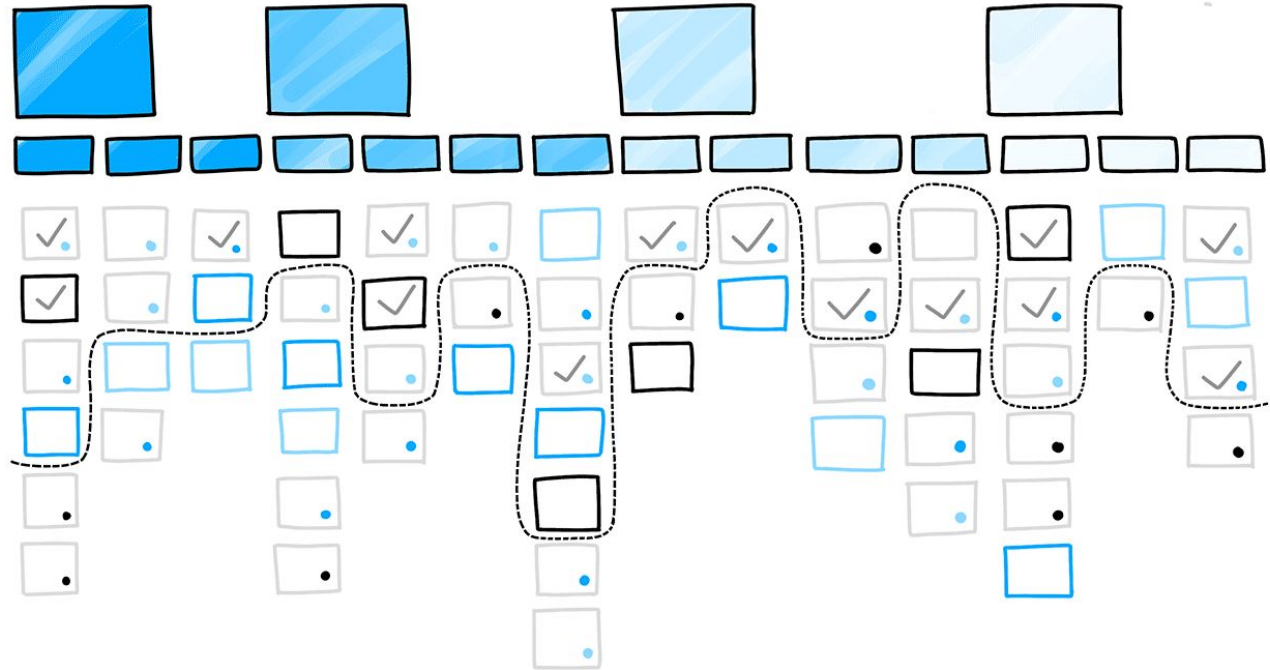
Snakes & Ladders



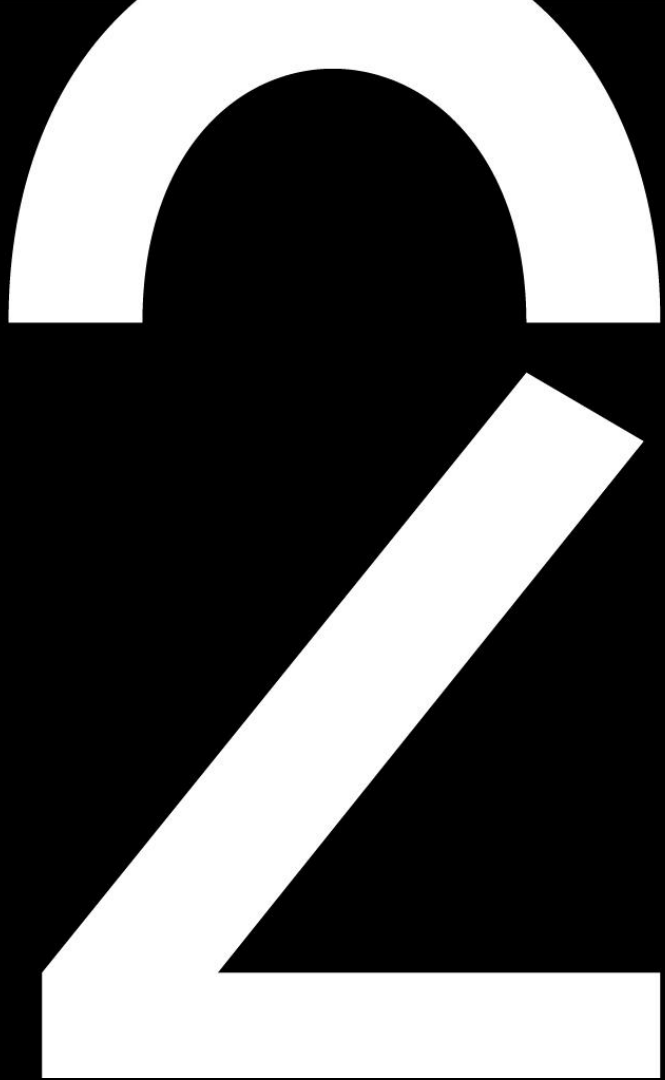
How to do Story mapping?

6. Define iterations

- a. Cut vertically through all tasks/user stories to determine release boundaries.



Wireframes



What is a wireframe?

A wireframe is a low-to-mid fidelity user experience design layout of a website, app, or software product—such as a building blueprint—that serves several purposes:

- It presents the information that will be displayed (pages, screens, displays,...)
- It provides an overview of the structure and layout of the pages, screens, displays...
- It gives an idea of how the navigation on the website, app or software product will proceed, what the interaction with the user will be like.

Without the distraction of colors, fonts, or text, wireframing allows you to plan the layout and interaction of your interface.

Wireframes are the responsibility of the UX/graphic designer, but the goal is to get everyone involved in the project (client/customer/user, project manager, architect, user experience team, QA, infrastructure, etc.) on the same page.

What is a wireframe?

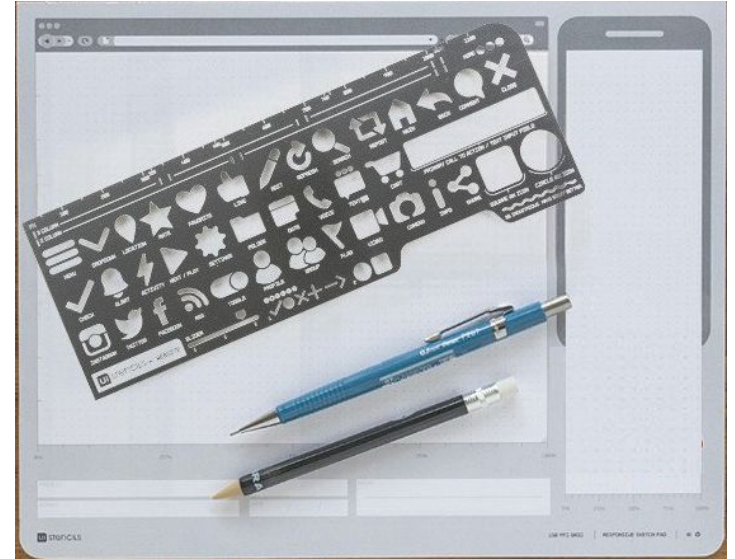
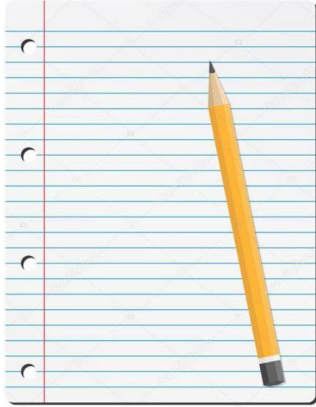
A wireframe is a step in the design process:

Sketch > wireframe > mock-up > prototype



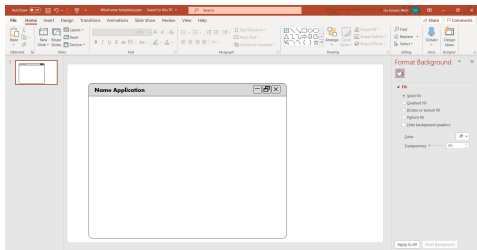
Tools

Pencil & paper Pencil, paper & stencil Sketch pad, pencil & stencil



Tools

Powerpoint



downloadable software

“Free” software

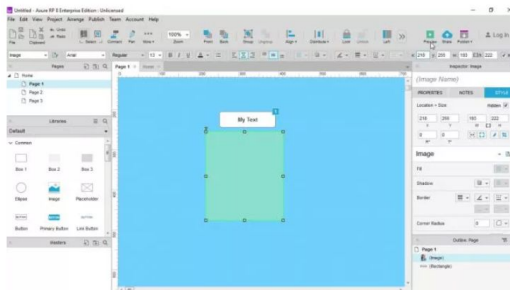
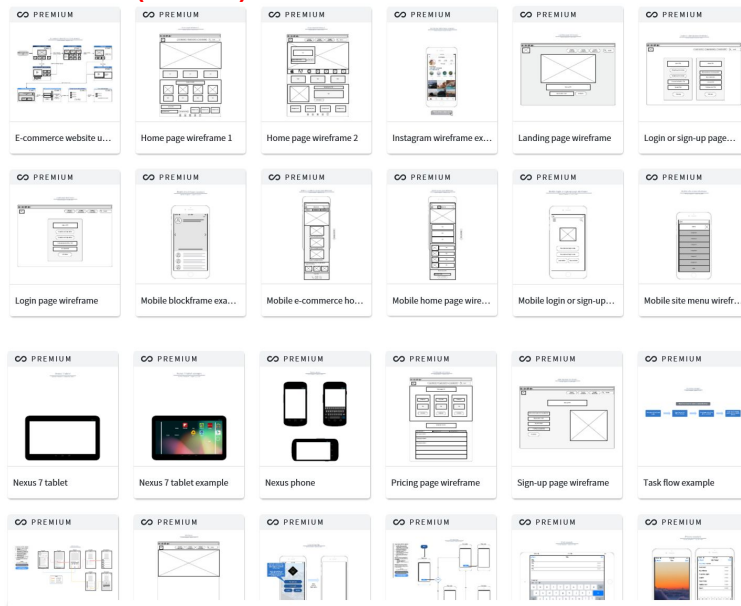
Figma
Adobe Xd (basic)
InVision
Miro (basic)
MockFlow (basic)
Moqups(basic)
OmniGraffle
Pencil Project
Pidoco
UXPin (basic)
Wireframe.cc (basic)
...

Purchase or subscription

Balsamiq Mockups
Fluid UI
Gliffy
HotGloo
InDesign CC
Justinmind
Photoshop CC
Protoshare
Visio
...

online software

Free (limited):



How do you make a wireframe?

Step 0 - Preparation

Before you start creating wireframes:

- Conduct user research (= understand who your audience is)
- Creating user personas (= fictional representation of your ideal customer(s))
- Creating use cases or user stories (= ways to list detailed requirements)
- Conduct competitive research (analyze similar product lines of your competitors)
- Conducting industry research (digging into current UX trends and best practices,...)



How do you make a wireframe?

Step 1 - Number of screens

Determine:

- How many screens do you need?
- The flow the user must follow between screens



How do you make a wireframe?

Step 2 - Sketching features and sizes

- Sketch the functions and shape with the following questions in mind:
 - How can you organize your content to support your users' goals?
 - **Which information** should be most prominent?
 - Where should the main message go?
 - What should the user see first when they arrive on the page?
 - What does the user expect to see on certain parts of the page?
 - Which menus, buttons, touchpoints... does the user need to complete the desired actions (call to action)?

Step 3 - Wireframing Together

- Discuss the wireframe(s) with peers.
- Play with the wireframe components. Reposition them on the screen/display.
- Answer the question "Are we creating something useful that meets the needs of our audience?"

How do you make a wireframe?

Step 4 - Adding Details and Highlights

- Add usage conventions (e.g. top navigation, your logo, a search box at the top right, etc.)
- Adding highlights (= clarifications, e.g. instructional wording for calls to action,...)
- Add header, menu, footer, ...
- Connect all wireframes together to create a flow

Step 5 - User Testing

- Discuss the wireframe with the client/customer/user/stakeholders
- Adjust the design

Tips:

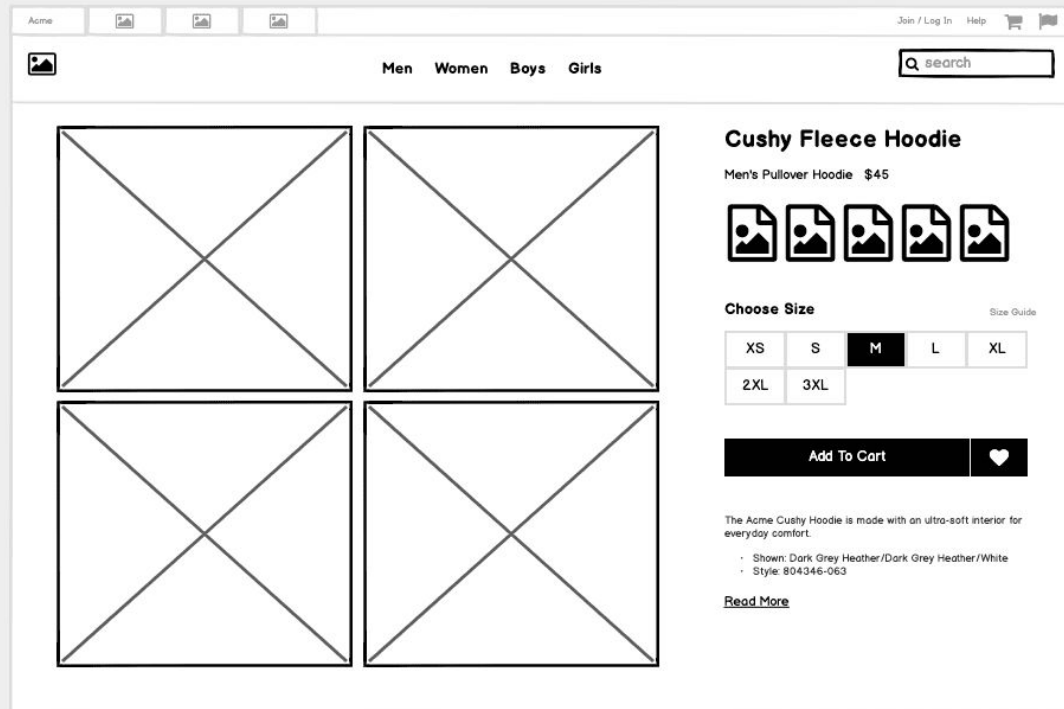
Aim for clarity and simplicity

Don't think about aesthetics, don't think about colors, fonts,...

Wireframes evolve as insights progress

Example

Product View



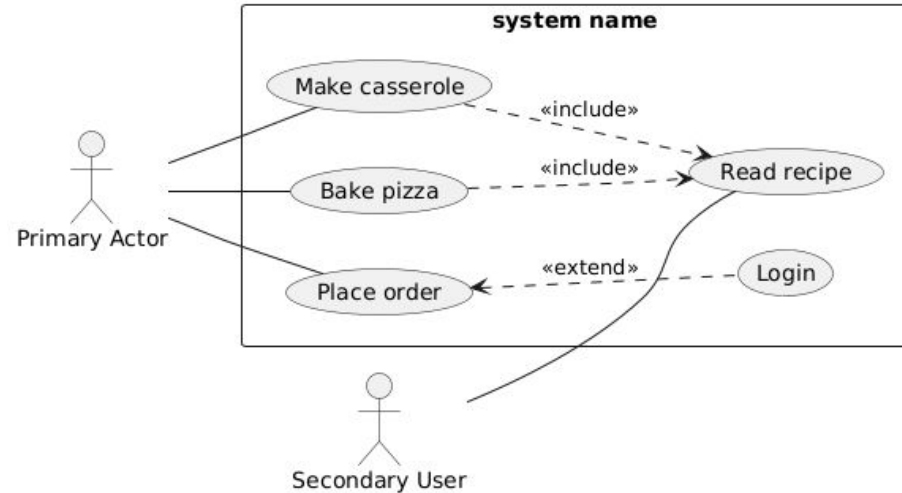
Use Case Diagram: General

Use Case Diagram

A use case diagram (UCD) is a graphical representation of the possible interactions of a user with a system.

A use case diagram shows use cases and types of users that the system has.

The use cases are represented by ellipses, the users by sticky-people. (Wikipedia)

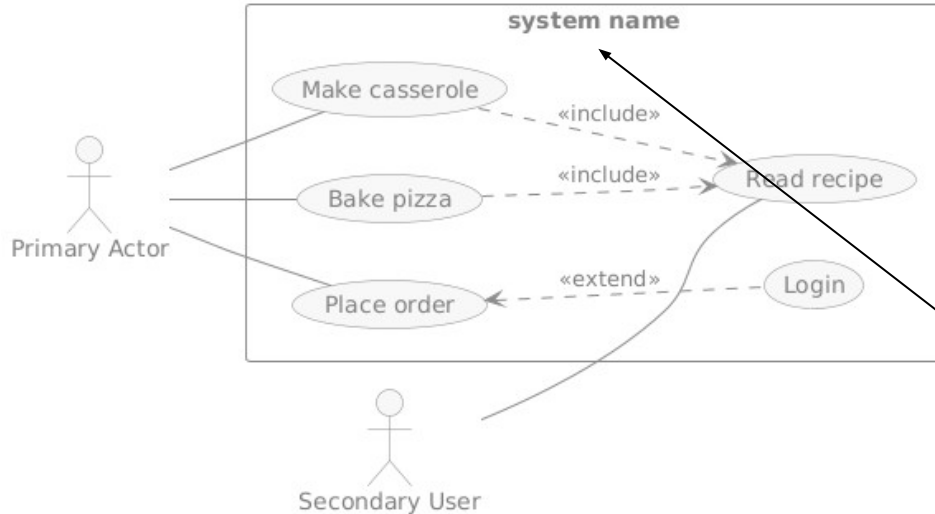


Use Case Diagram: All parts explained

System

UCD shows interactions between users and the system. Therefore, it's important to define the system boundaries.

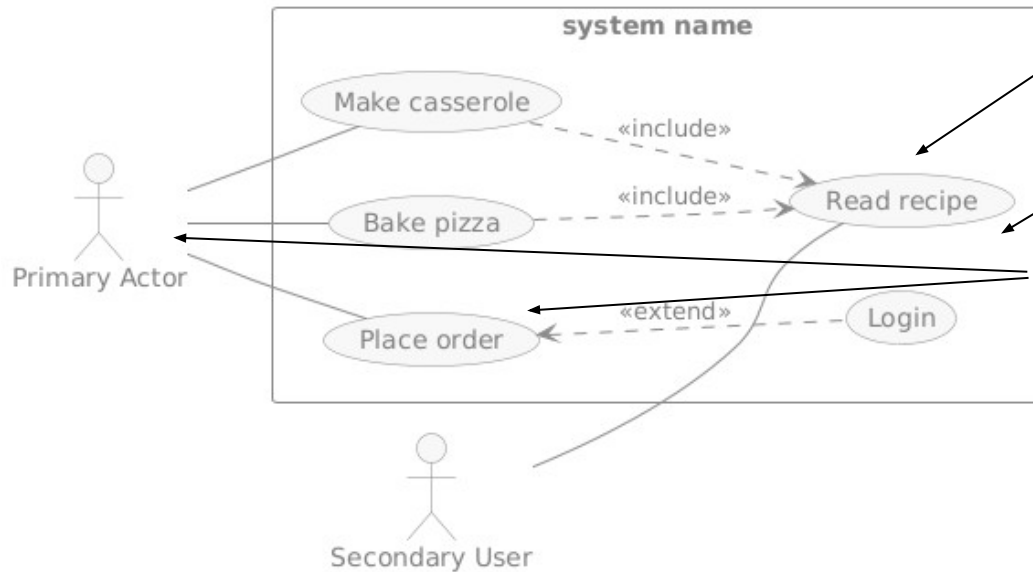
- rectangle - the boundaries of the system. Everything inside → the (software) system that needs to be built
- name - at the top we write the name of the system



Use Case Diagram: All parts explained

Use Case

The 'function' of the system
(process - automated or manual)

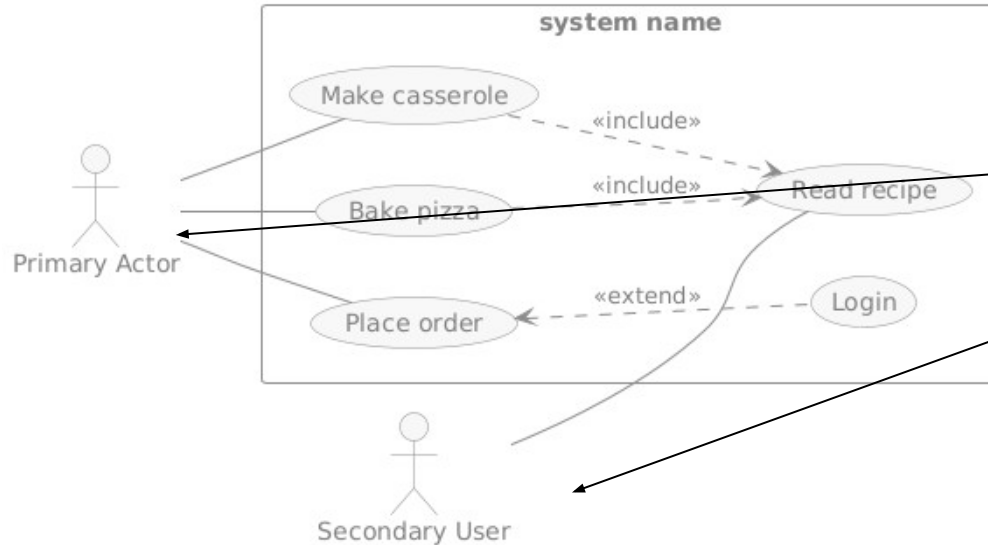


- Ellipse: Indicates that it concerns a Use Case
- Text: use case name: verb+object
- Link: On the left, each Use Case must be linked either to an Actor or to another use case
- Optional links to show relation between use cases
 - include: ALWAYS performed
 - extend: OPTIONALLY performed

Use Case Diagram: All parts explained

Actor

The 'user' of the system, this can be a human (sticky-men), or another system or event.



- Left side: primary actor - 'starts' the use case
- Right or bottom: secondary actor
 - Supporting - also takes actions that are necessary
 - Offstage - receives something as a result of the Use Case

Use Case Diagram: Points of interest

- No assumptions: Only mention the use cases described in the Business Case, don't make assumptions about how you think it will work.
- Be complete: Make sure you include all Use Cases from the Business Case, read carefully line by line!
- Use Cases are always in the form: Verb DirectObject or DirectObject Verb
 - e.g. Pay for order or Create exam

Use Case Diagram: Points of interest

- There may be inheritance of the actors, but...
 - Then ALL use cases of the SuperActor must also apply to the SubActor in exactly the same way
 - Sometimes it's better to take over a role. For example, a teacher takes on the role of student to administer an exam. "Administer exam" is NOT a use case for teacher in that case.
- "include": this must ALWAYS be done
- "extend": this can also be done OPTIONALLY
- Place the arrows of "include" and "extend" in the reading direction
 - e.g. 'log in' extends 'change data' : 'change data' ← 'log in'
 - e.g. 'create questionnaire' includes 'create question' : 'create questionnaire' → 'create question'

Sample exercise: assignment

Business Case

The restaurant has a whole team of waiters. Each junior waiter is supervised by a senior waiter. A senior waiter can supervise several junior waiters.

When you arrive at the restaurant, your visit will be registered by someone at the entrance and a waiter will be assigned to your guest who will be responsible for taking orders.

For each customer order, a record is kept of which waiter takes the order and which waiter serves the dishes. This can also be a different waiter each time. These orders then go to the kitchen where they are ultimately prepared.

Only the most experienced senior waiter may settle your bill. The bill will then be processed through a payment service.

Customers usually make reservations by phone. Any restaurant employee can enter the reservation in the system. Customers will then receive a confirmation text message.

At the end of the day, the senior will also close the cash register.

Example exercise: solution

Business Case

When you arrive at the restaurant, your visit will be registered by someone at the entrance and a waiter will be assigned to your guest who will be responsible for taking orders.

visit registered : use case name : register visit / register visit actor : unclear - 'someone at the entrance' → employee

waiter assigned : use case name : assign waiter actor : there is no actor, this use case is ALWAYS executed on arrival registering visit gives rise to waiter assignment : "includes"

take order : use case name : take order actor : Waiter

Example exercise: solution

Business Case

For each customer order, a record is kept of which waiter takes the order and which waiter serves the dishes. This can also be a different waiter each time. These orders then go to the kitchen where they are ultimately prepared.

takes order: Use case name: take order (same as previous page) Actor: Just because the customer places the order doesn't mean they enter it into the system. We just discovered that waiters take orders. In other words, the customer is not a primary actor. We can say that the customer is an OFFstage actor (she receives food).

order in kitchen : use case name : take order actor : kitchen - this is a Supporting Actor (necessary to complete order)

Example exercise: solution

Business Case

Only the most experienced senior waiter may settle your bill. Payment will then be processed through a payment service.

create bill : use case name : checkout visit actor : SENIOR waiter, apparently all waiters are allowed to take service, but only Seniors are allowed to pay : INHERITANCE needed

via payment service: use case name: checkout visit actor: payment service - this is a Supporting Actor (necessary to complete order), this is a SYSTEM - so please note that it should not be done with sticky men

Example exercise: solution

Business Case

Customers usually make reservations by phone. Any restaurant employee can enter the reservation in the system. Customers will then receive a confirmation text message.

At the end of the day, the senior will also close the cash register

?? : use case name : ?? actor : ???

Now try to find the use cases yourself!!

?? : use case name : ?? actor : ??

Example exercise: solution

Business Case

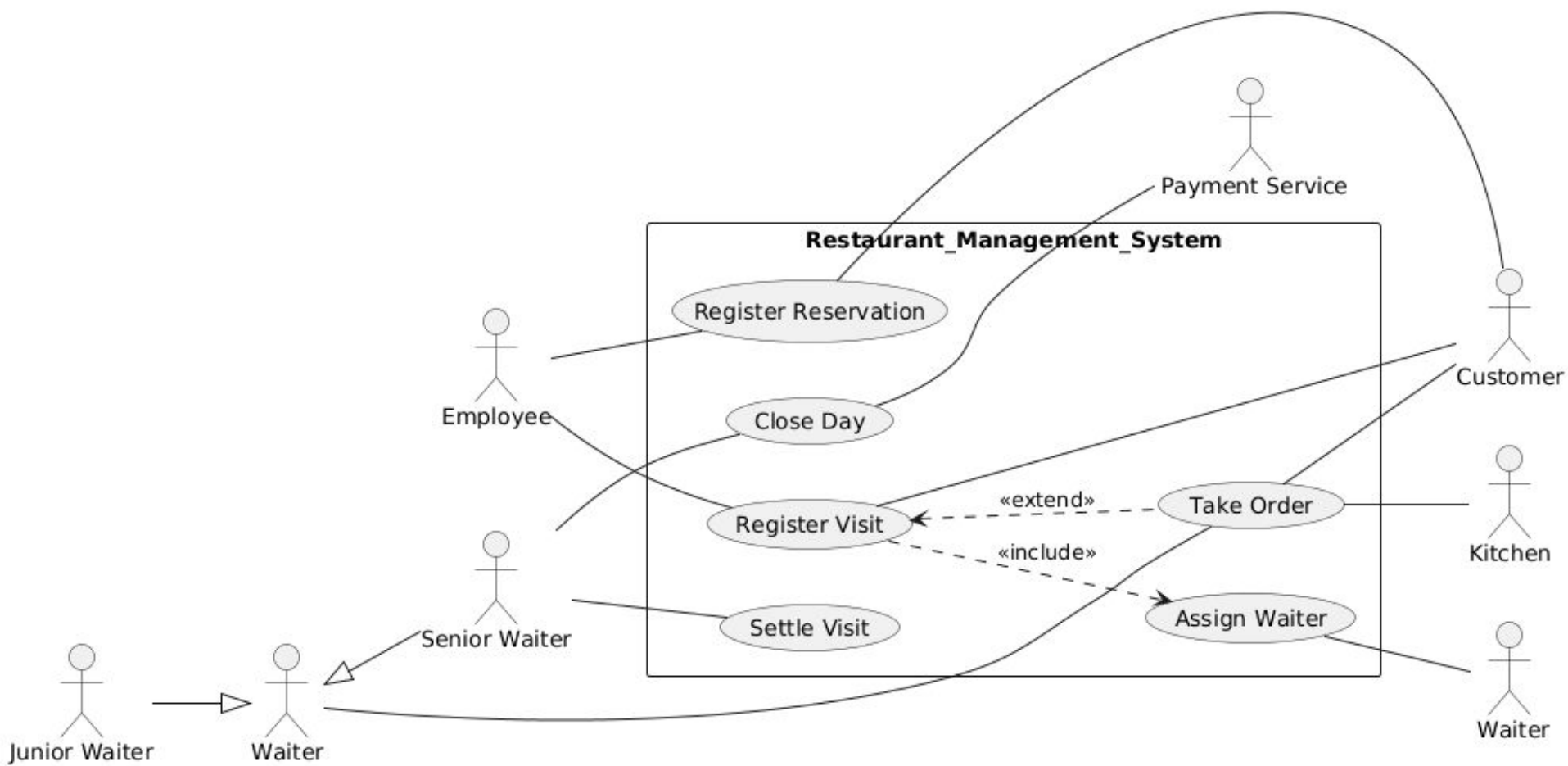
Customers usually make reservations by phone. Any restaurant employee can enter the reservation in the system. Customers will then receive a confirmation text message.

At the end of the day, the senior will also close the cash register

book a reservation: The customer interacts with an employee (who is outside the system), NOT with the system directly → so this is not a primary actor, nor a use case

register reservation: use case name: register reservation actor: primary is employee, offstage is customer (receives an SMS)

close cash register : use case name : close cash register actor : senior waiter



Example exercise: solution

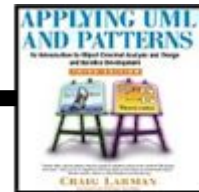
some more comments

- You don't see any difference between offstage and supporting actors on the right - we'll include that information in the Use Case description (see last week)
- register visitor -extends → Take order This link is not LITERALLY in the text, but we can infer it from the fact that registering a visit leads to the assignment of a waiter. Only waiters are allowed to take orders. So... before the waiter takes the order, the visitor may first have to be registered. PS. This is 'advanced' - on the exam it will not be so indirect
- Junior Waiter - we know they exist, but there is no single Use Case specifically for them, so they do NOT need to be on the diagram.

Activity Diagram

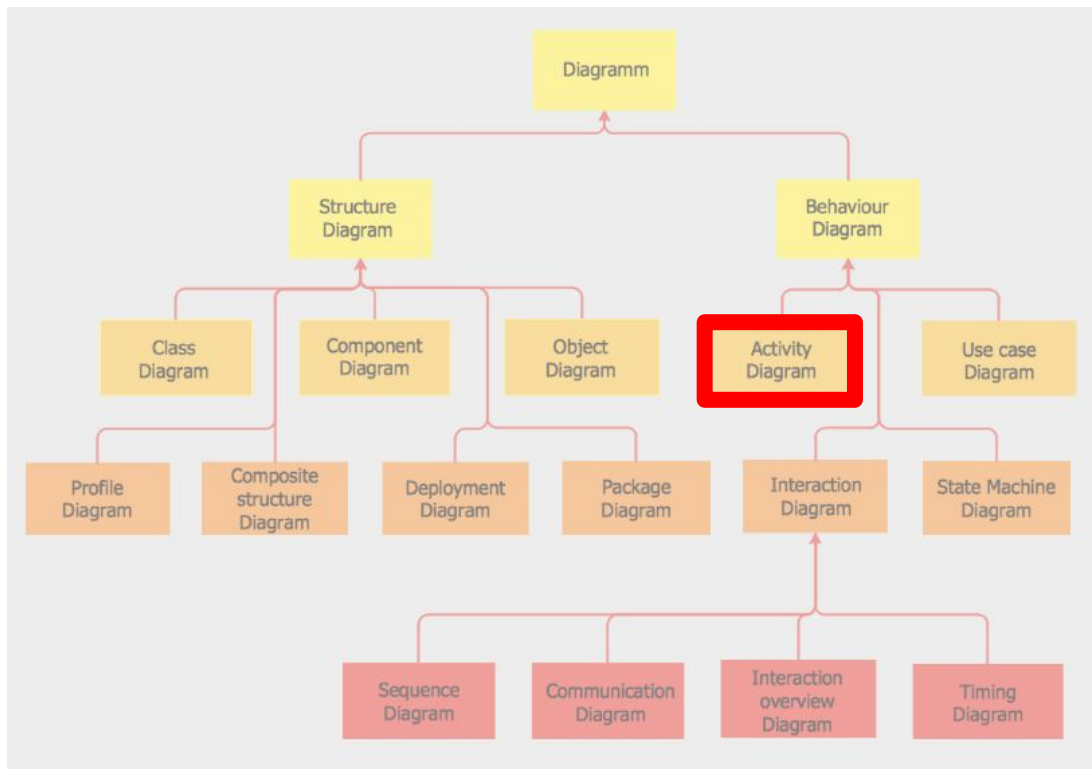


Introduction



10.4

UML (Unified Modelling Language) defines 14 official diagram types, divided into 2 major categories:

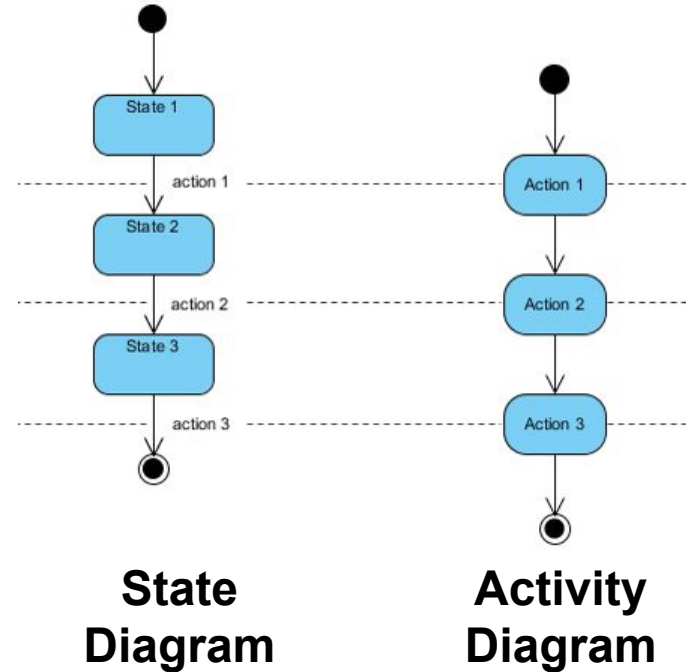


Source: UML Diagram Types List

Activity vs State Diagrams

Activity Diagrams = Activity diagram (shows actions)

State (Machine) Diagrams = State diagram (shows state) (of machine/controller)



Introduction

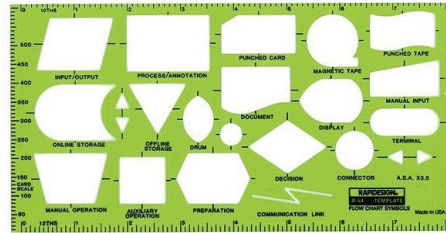
Activity Diagrams can be used at different levels:

- Business Process / Workflow
 - Procedure for applying for special status for a student
 - Procedure for allocating a company car
- System Use Case
 - steps in scenario (usually main success + alternative scenarios)
- Method
 - steps in a method

No description of the structure.

Introduction

Once upon a time... flowcharts.



Activity diagrams:

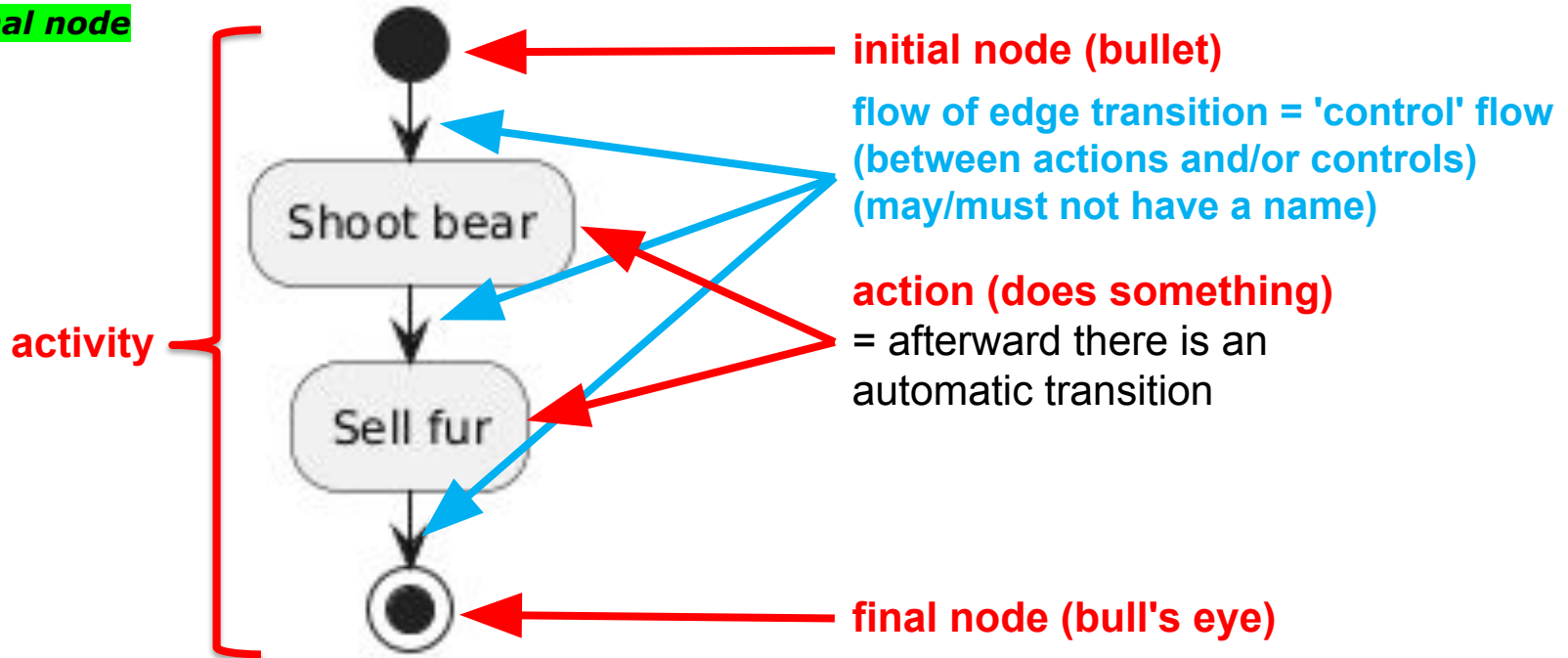
- UML compliant flowchart
- Addition of parallelism
- Addition of loops
- Further standardization

□ UML as a programming language (behavioral logic)

Activity & action

An activity is a sequence of:

- **initial node**
- one or more actions (nodes)
- **final node**



Activity & action

action

- step that can be performed
- notation: rectangle with rounded corners



flow/edge

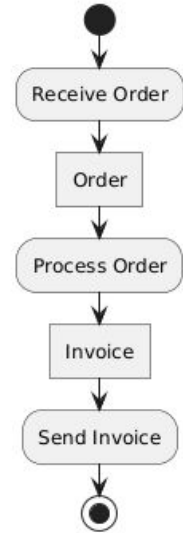
- General: From node to node, there are 3 types of activity nodes: action node (rectangle with rounded corners), object node (regular rectangle), and control node (sphere, diamond, or thick line).
- for an action applies: exactly one incoming and one outgoing flow (transition)
- notation: solid line with open arrowhead



UML1 and UML2 disagree on the interpretation of multiple incoming flows for the same action, so we don't use it! See below...

Object node

- This is the standard UML notation for an object node:
- **Shape: Regular rectangle with right angles**
- **Content: usually the name of the object and/or the object type**
- **Usage: between activities to indicate data flows**
- Object node:
 - **object = (certain state of) object of class**
 - **datastore = data storage, no transitions (control flow) but data flow... cf. data flow diagrams (DFD)**



Example: Order and Invoice are object nodes, which represent concrete instances of the Order and Invoice classes.

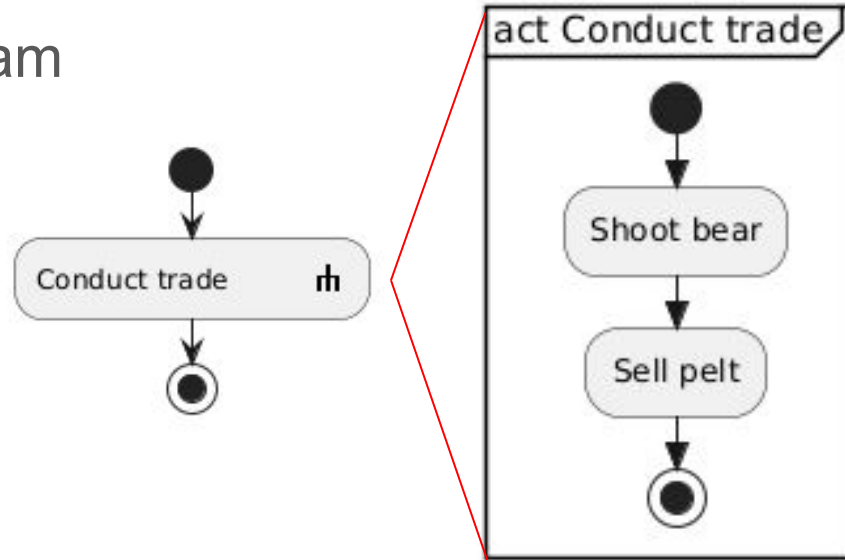
- In the activity diagram, you see object nodes as objects (e.g., a specific order, a specific invoice) that move through the process.
- The Order and Invoice classes are the blueprints, the types from which these objects are derived.
- During the execution of the process, those objects are modified, updated or used by the activities (such as "Process Order").

Activity & action

subactivity

an action that is described
in another activity diagram

Notation with a "rake"

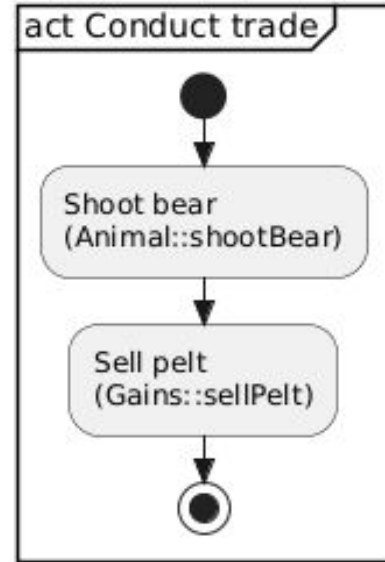


Activity & action

If an action matches a class method this can be indicated

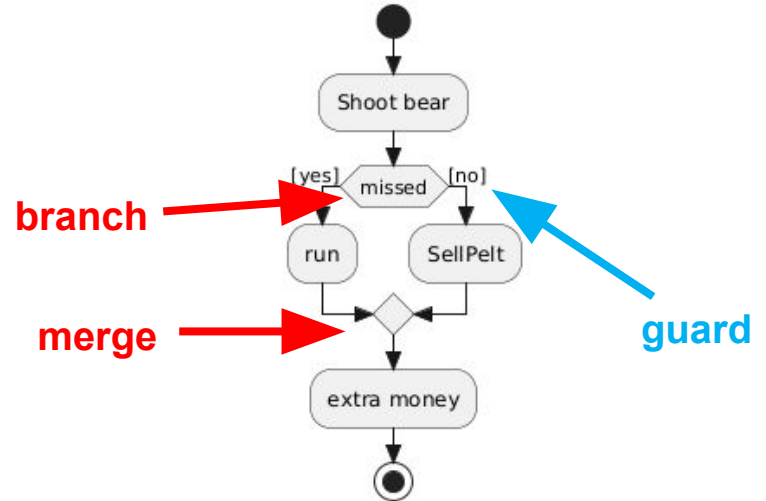
Notation in parentheses with “::”

(Classname::methodName)



Conditional behavior

- **decision of branch node**
 - only one incoming flow
 - multiple outgoing flows with guard (condition)
- **guard**
 - mandatory for every outgoing flow
 - mutually exclusive
 - between [and]
 - if necessary, use [else]
- **merge node**
 - multiple incoming flows
 - only one outgoing flow
 - never guards...

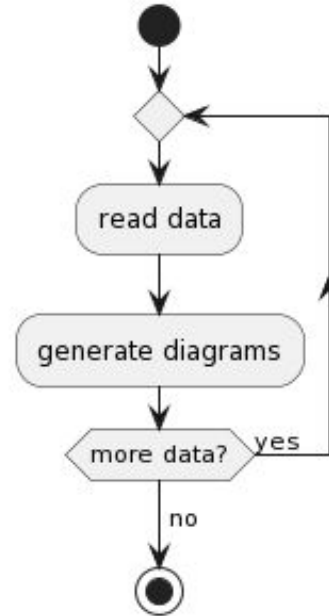


Conditional behavior

Repetitions (loops) possible.

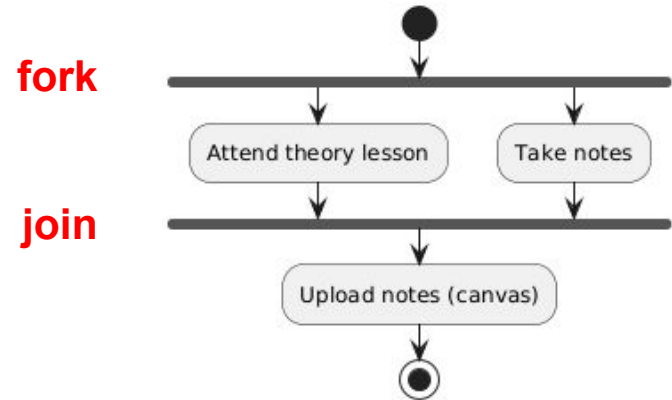
Types of loop...

- While (condition) { Do }
- Repeat { } Until (condition)
- For (condition)



Parallel behavior

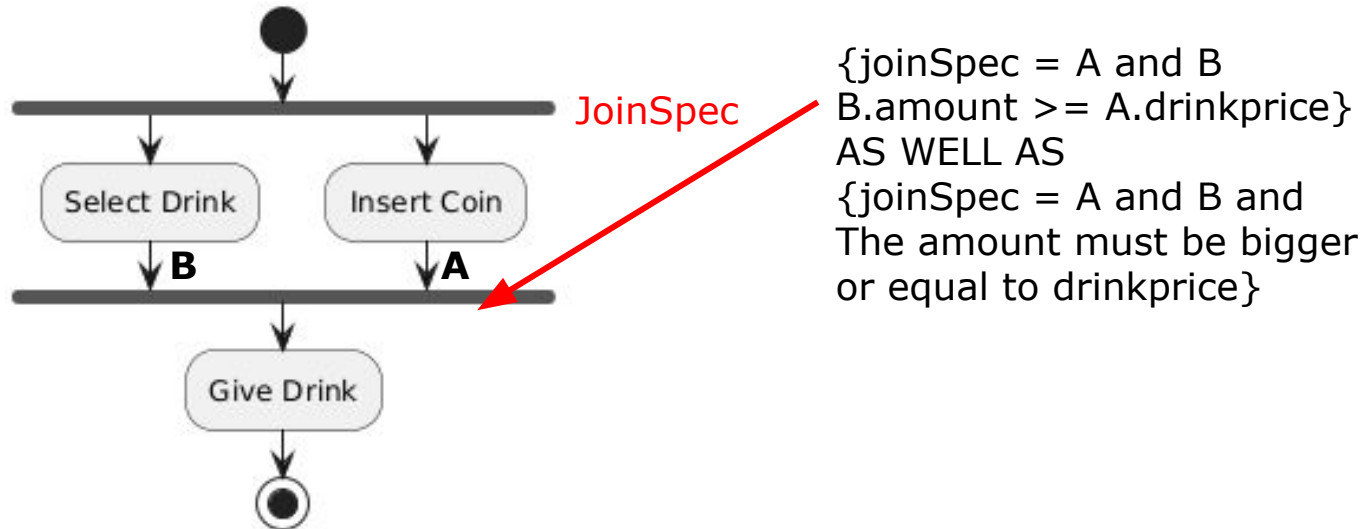
- Possibly by one actor or application (e.g. multi-threading)
 - Possibly by different actors (e.g. waiting for input from another actor)
 - Synchronization bar = thick horizontal or vertical line
-
- fork node = split
 - join node = synchronization
-
- between the parallel flows
the order is not relevant



**can also be drawn
vertically!**

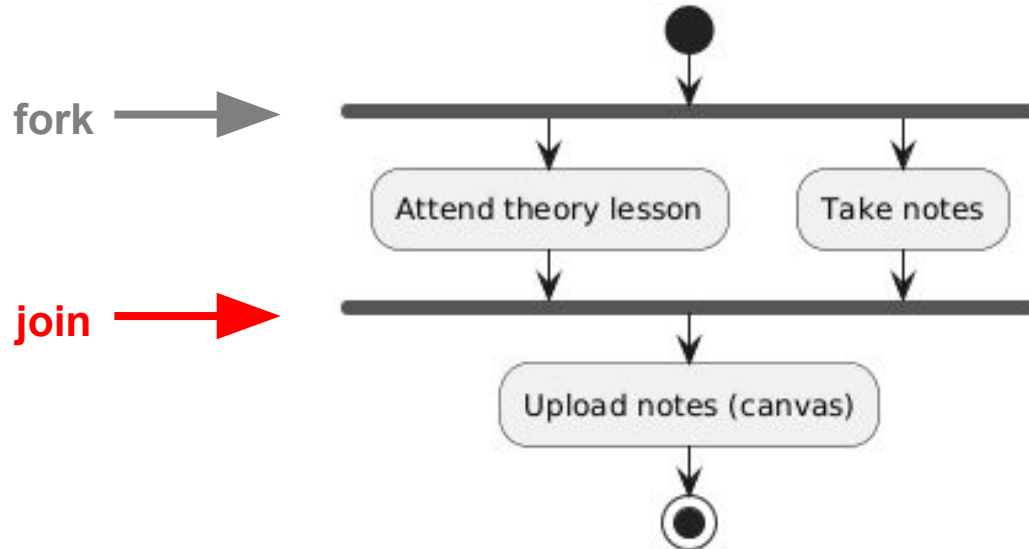
Parallel behavior - join specification

- A join specification is a logical condition on a join label
- Any tokens that do not meet the condition will be destroyed by this join specification
- If you see a need to combine an AND with an OR in a join specification, you're better off using a decision node.

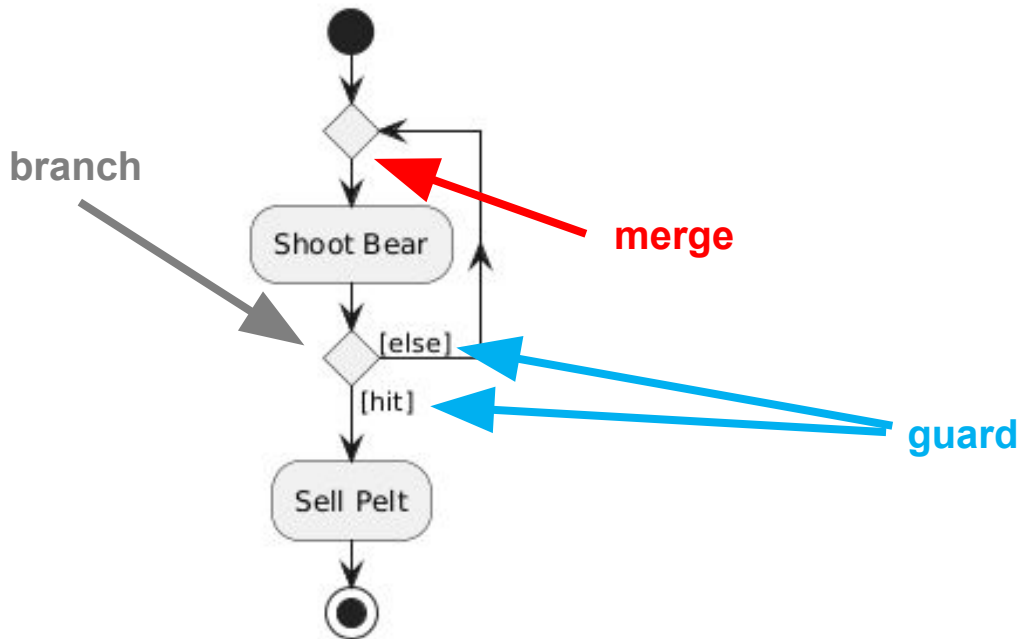


Parallel behavior

- UML1 and UML2 disagree on the interpretation of multiple incoming flows for the same action, so we don't use it...
 - UML1 -> (implicit) merge (conditional behavior)
 - UML2 -> (implicit) join (parallel behavior)
- Agreement: we always write merge and join explicitly

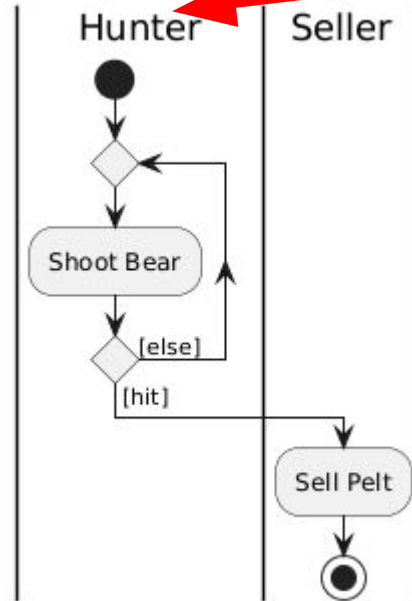


- Agreement: we always write merge and join explicitly



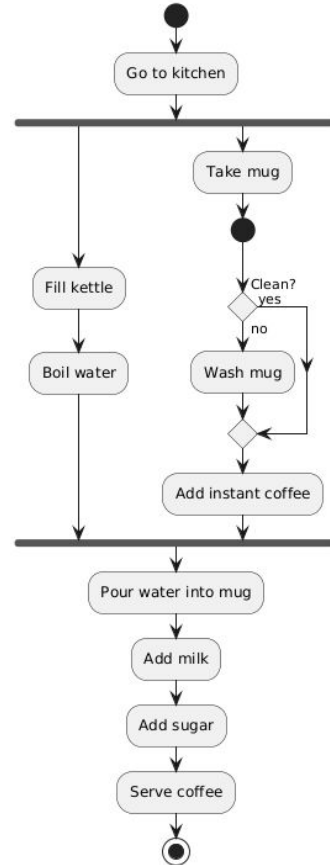
Activity partition: Swimlane

- Split with lines vertically or horizontally = swimlane
- At the top we note who is responsible for what = which actions (actors, classes, objects, attributes,...)



Example 1 (making instant coffee in the kitchen)

Activity Diagram - Making Instant Coffee



Applications

- Activity Diagram for workflow => business process, workflow, BPMN
- Activity Diagram as an algorithm => system use case

Recurring Example: Monopoly

Purpose of the Exercise

This exercise is an ongoing case study in Conceptual Thinking. Each week, we apply what we've learned to a digital version of Monopoly.

Why do we choose Monopoly?

- Everyone knows it, or almost everyone
- It contains rich structures: players, money, rules, actions, spaces, cards
- It provides an ideal context for modeling object-oriented systems
- It is tangible and recognizable, making abstract concepts concrete



Recurring Example: Monopoly

Learning objective

Students understand how theory is translated into a working conceptual model by:

- Working iteratively with one consistent case
- Apply different analysis methods (use cases, class diagrams, ...)
- Gain insight into the relationship between models and system behavior

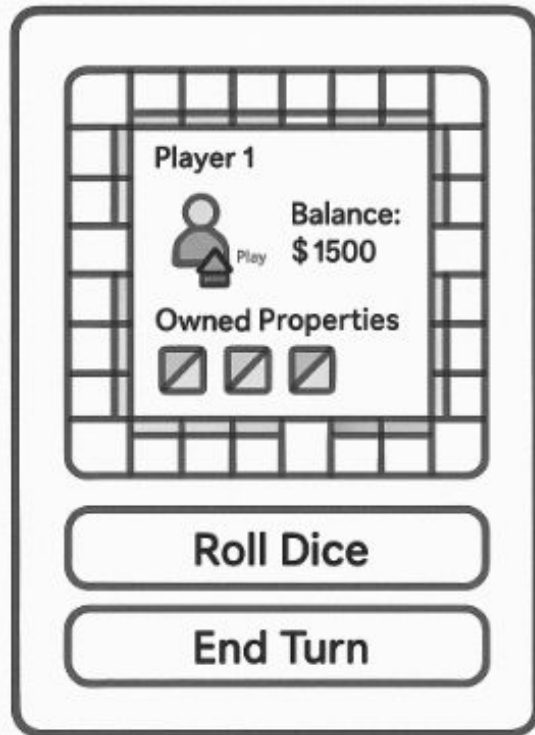
Quick Refresher: How Does Monopoly Work Again?

- 2–6 players move their pawn around the board with a roll of 2 dice
- Players buy properties they land on
- Other players pay rent when they land on someone else's property
- There are special subjects such as Chance, Community Fund, Tax, Prison, etc.
- Players can take out mortgages, trade properties, or go bankrupt
- Goal: Become the richest player by bankrupting others

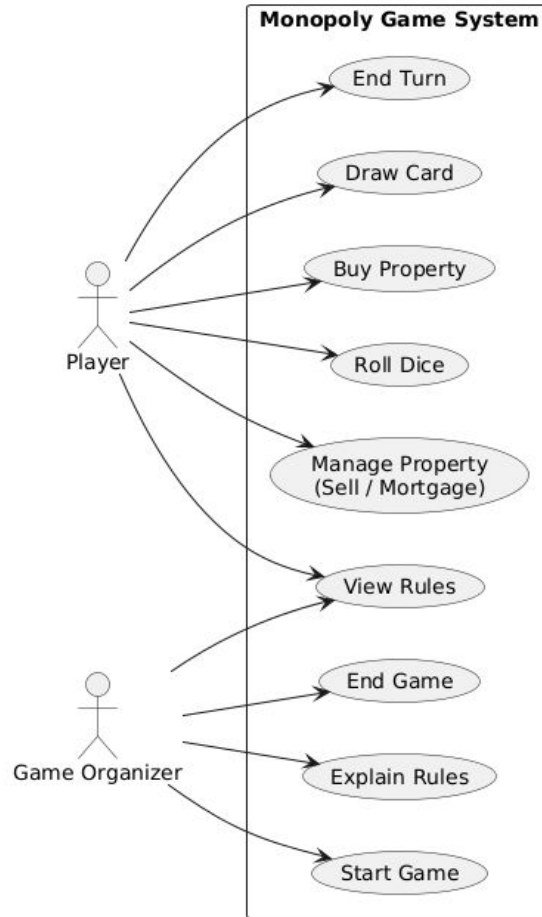
Storymapping

Activity	User Stories
Start game	- As a player, I want to be able to start the game so that everyone can participate. - As an organizer, I want to be able to show the rules before the game starts.
Take a turn	- As a player, I want to be able to roll dice to move my pawn.- As a player, I want the effects of the field I land on to be carried out automatically.- As a player, I want to get an extra turn when I roll a double.- As a player, I want to automatically go to jail when I roll a double three times.
Manage property	- As a player, I want to be able to buy an empty property when I land there. - As a player, I want to be able to sell or mortgage a property. - As a bank, I want to automatically withdraw the player's money when purchasing. - As a bank, I want to manage properties that have no owner.
Consult rules	As a player, I want to be able to review the rules during the game. As an organizer, I want to be able to explain the rules to new players.
Draw cards	- As a player, I want to automatically draw a card when I land on a Chance or Community space. - As a system, I want to automatically apply the card's effect.
End game	- As a system, I want the game to end automatically when there is only one player left. - As an organizer, I want to be able to end the game manually.
Prison scenario	- As a player I want to have options to get out of jail (pay, card, dice).
Interface & UX	- As a player, I want clear visual feedback on actions. - As a user, I want to be able to use the app on both desktop and tablet.
Synchronization	- As a system, I want to keep every turn automatically synchronized across all player screens.
System management	As an admin, I want to be able to add new card instructions without writing code. As a system administrator, I want to be able to log game actions for debugging or replay.
System reliability	- As a system, I want to preserve the game state in case of disconnection. - As a system, I want to execute transactions correctly and without duplication. - As a system, I want to guarantee 99.8% uptime during gaming sessions.

Wireframes

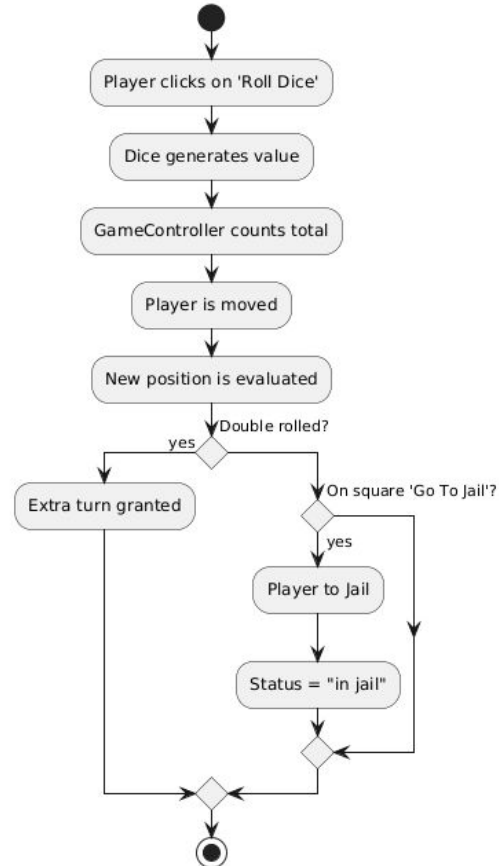


Use case diagram



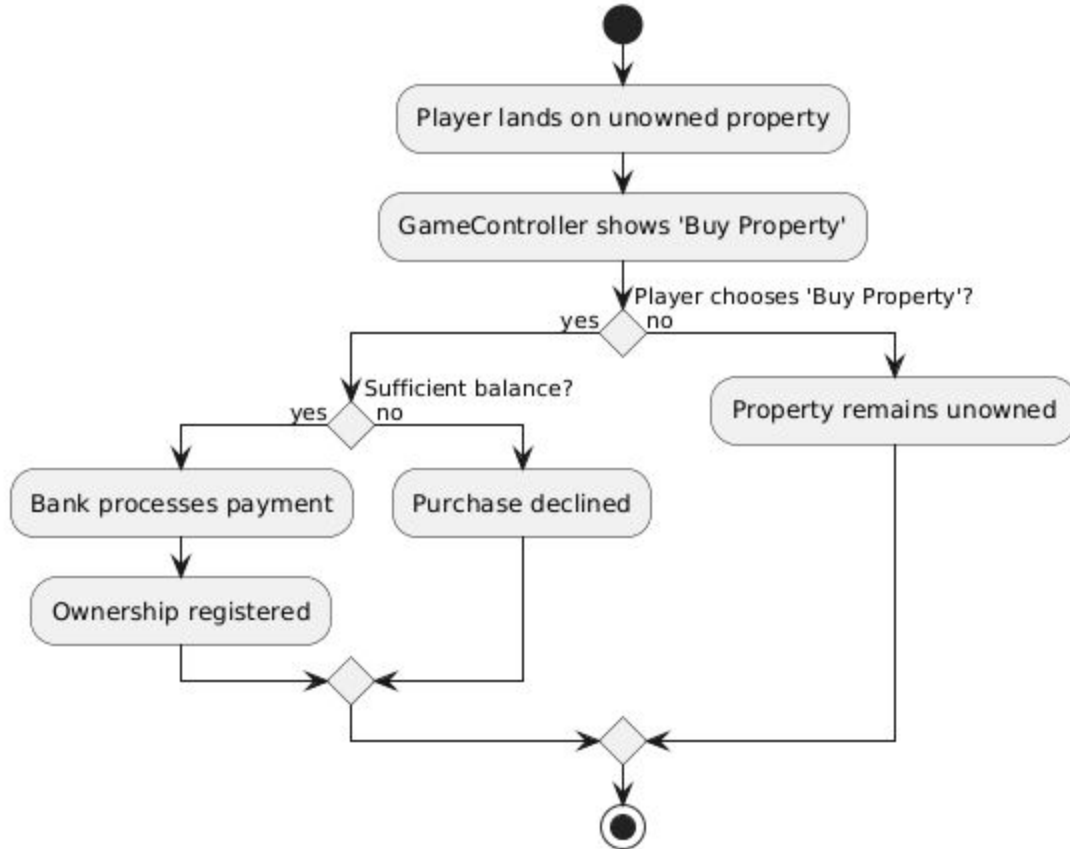
Activity diagram

Activity Diagram - Roll Dice



Activity diagram

Activity Diagram - Buy Property



Activity diagram

Activity Diagram - Explain Rules

